River Sizes

AlgoExpert · Medium · Graphs

The problem

- In a 0/1 matrix, sizes of all 4-directionally connected groups of 1s (any order).

Intuition

- Each river is a connected component of ones; flood-fill to measure it.

Brute force vs optimal

- Brute force: re-explore from every 1 → repeated work
- Visited matrix + DFS/BFS → each cell once, O(w·h)
- Iterative stack avoids recursion limits

Approach

- 1. Scan every cell
- 2. On an unvisited one, DFS or BFS to count its size
- 3. Mark visited cells

Worked example




```
sizes:

[2]
```

River at (0,0) → size 2

Worked example

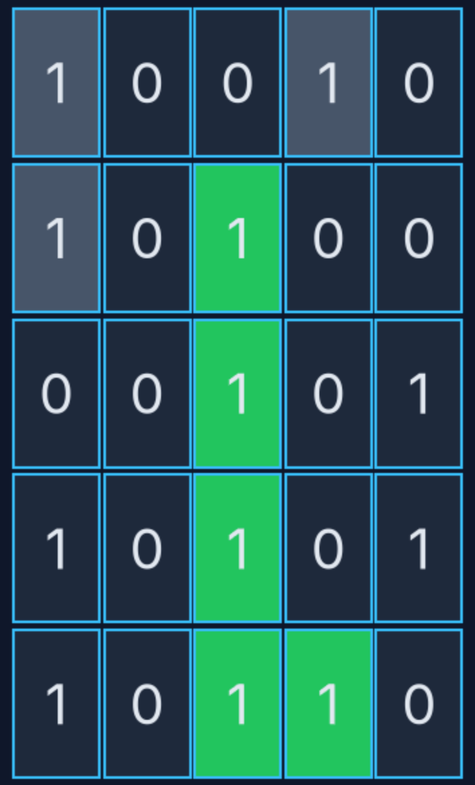
1	0	0	1	0
1	0	1	0	0
0	0	1	0	1
1	0	1	0	1
1	0	1	1	0

```
sizes:

[2, 1]
```

River at (0,3) → size 1

Worked example




```
sizes:

[2, 1, 5]
```

River at (1,2) → size 5

Worked example

1	0	0	1	0
1	0	1	0	0
0	0	1	0	1
1	0	1	0	1
1	0	1	1	0

```
sizes:

[2, 1, 5, 2]
```

River at (2,4) → size 2

Worked example

1	0	0	1	0
1	0	1	0	0
0	0	1	0	1
1	0	1	0	1
1	0	1	1	0

```
sizes:

[2, 1, 5, 2, 2]
```

River at (3,0) → size 2

Java solution

```java
public class RiverSizes {
    public static List<Integer> riverSizes(int[][] m) {
        List<Integer> sizes = new ArrayList<>();
        boolean[][] seen = new boolean[m.length][m[0].length];
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        for (int r = 0; r < m.length; r++)
            for (int c = 0; c < m[0].length; c++) {
                if (m[r][c] != 1 || seen[r][c]) continue;
                int size = 0;
                ArrayDeque<int[]> st = new ArrayDeque<>();
                st.push(new int[]{r, c});
                seen[r][c] = true;
                while (!st.isEmpty()) {
                    int[] p = st.pop();
                    size++;
                    for (int[] d : dirs) {
                        int nr = p[0] + d[0], nc = p[1] + d[1];
                        if (nr >= 0 && nc >= 0 && nr < m.length && nc < m[0].length && m[nr][nc] == 1 && !seen[nr][nc]) {
                            seen[nr][nc] = true;
                            st.push(new int[]{nr, nc});
                        }
                    }
                }
                sizes.add(size);
            }
        return sizes;
    }
```

Complexity

- Time: O(w*h)
- Space: O(w*h)

Edge cases & pitfalls

- Mark visited when pushing, not popping
- 4-directional adjacency
- Empty grid / no rivers → []

Interview tips & follow-ups

- Mention union-find alternative
- Follow-up: largest island after flipping one 0 (Largest Island)

Recap

- Each river is a connected component of ones; flood-fill to measure it.
- Complexity: O(w*h)